Heston Stochastic Volatility Modelling

Motivation

The Black-Scholes model, whilst foundational to quantitative finance, assumes constant volatility—an assumption that fails to capture the empirically observed volatility clustering, mean reversion, and leverage effects present in financial markets. The Heston stochastic volatility model addresses these limitations by introducing a stochastic process for the variance that exhibits mean reversion and allows for correlation between asset returns and volatility movements.

In practice, the Heston model provides a flexible framework for pricing European and exotic options, calibrating to market-implied volatility surfaces, and capturing the volatility smile phenomenon. Unlike simpler models, the Heston framework enables quantitative analysts to incorporate realistic volatility dynamics whilst maintaining analytical tractability through its closed-form characteristic function.

Monte Carlo simulation under the Heston model requires careful consideration of numerical methods. Standard Euler-Maruyama discretisation can produce negative variance paths, violating the non-negativity constraint inherent in variance processes. Exact simulation methods, based on the distributional properties of the variance process, eliminate this issue by sampling directly from the transition density of the variance, resulting in more accurate and computationally efficient option pricing.

This project implements an exact simulation scheme for the Heston model using the characteristic function inversion technique, enabling accurate pricing of path-dependent and exotic options whilst maintaining numerical stability. The interactive dashboard provides real-time analysis of volatility dynamics, option pricing sensitivities, and Monte Carlo convergence behaviour, making it an invaluable tool for quantitative research, risk management, and educational purposes.

Theory

This project implements a Monte Carlo pricing framework for the Heston stochastic volatility model using exact simulation methods to ensure numerical stability and accuracy.

The Heston Stochastic Volatility Model

Definition (Heston Model Dynamics)

The Heston stochastic volatility model describes the evolution of an asset price $S(t)$ and its instantaneous variance $v(t)$ under the risk-neutral measure:

$$\begin{aligned} \mathrm{d}S(t) &= rS(t)\,\mathrm{d}t + \sqrt{v(t)}S(t)\,\mathrm{d}W_S(t), \\ \mathrm{d}v(t) &= \kappa(\theta - v(t))\,\mathrm{d}t + \sigma_v\sqrt{v(t)}\,\mathrm{d}W_v(t), \end{aligned}$$

where r is the risk-free interest rate, $\kappa > 0$ is the mean reversion speed, $\theta > 0$ is the long-term variance level, $\sigma_v > 0$ is the volatility of volatility, and $\mathrm{d}W_S(t)$ and $\mathrm{d}W_v(t)$ are correlated Brownian motions with correlation $\rho \in [-1, 1]$.

The key characteristics of the Heston model include:

- **Mean reversion**: The variance process reverts to its long-term level θ at rate κ.
- **Feller condition**: To ensure $v(t) > 0$ almost surely, the condition $2\kappa\theta \geq \sigma_v^2$ must be satisfied.
- **Leverage effect**: The correlation ρ between asset returns and variance changes captures the empirical observation that volatility tends to increase when asset prices fall.
- **Volatility clustering**: The mean-reverting variance process naturally generates periods of high and low volatility.

Characteristic Function and Option Pricing

Theorem (Heston Characteristic Function)

The characteristic function of the log-asset price $X(t) = \ln S(t)$ under the Heston model is given by:

$$\phi(u,t) = \mathbb{E}\left[e^{iuX(t)}\right] = \exp\left(C(u,t) + D(u,t)v(0) + iuX(0)\right),$$

where

$$C(u,t) = riut + \frac{\kappa\theta}{\sigma_v^2}\left[(\kappa - \rho\sigma_v iu - d)t - 2\ln\left(\frac{1-ge^{-dt}}{1-g}\right)\right],$$

$$D(u,t) = \frac{\kappa - \rho\sigma_v iu - d}{\sigma_v^2} \cdot \frac{1-e^{-dt}}{1-ge^{-dt}},$$

$$d = \sqrt{(\rho\sigma_v iu - \kappa)^2 + \sigma_v^2(iu + u^2)},$$

$$g = \frac{\kappa - \rho\sigma_v iu - d}{\kappa - \rho\sigma_v iu + d}.$$

This characteristic function enables semi-analytical pricing of European options using Fourier inversion methods, providing a benchmark for validating Monte Carlo implementations.

Exact Simulation of the Variance Process

Definition (CIR Process Distribution)

The variance process $v(t)$ follows a Cox-Ingersoll-Ross (CIR) process, whose transition density is a non-central chi-squared distribution. For a time step Δt, given $v(t)$, the distribution of $v(t+\Delta t)$ is:

$$v(t+\Delta t) = \frac{\sigma_v^2(1-e^{-\kappa\Delta t})}{4\kappa}\chi_d^2\left(\frac{4\kappa e^{-\kappa\Delta t}}{\sigma_v^2(1-e^{-\kappa\Delta t})}v(t)\right),$$

where $\chi_d^2(\lambda)$ denotes a non-central chi-squared random variable with $d = 4\kappa\theta/\sigma_v^2$ degrees of freedom and non-centrality parameter λ.

Exact simulation samples $v(t+\Delta t)$ directly from this distribution, ensuring $v(t+\Delta t) \geq 0$ and eliminating discretisation bias inherent in Euler-Maruyama schemes.

Exact Simulation Algorithm

For a time step Δt from t to $t + \Delta t$, the exact simulation proceeds as follows:

1. **Sample variance**: Draw $v(t+\Delta t)$ from the non-central chi-squared distribution as specified above.
2. **Sample log-asset price**: Given $v(t)$ and $v(t + \Delta t)$, integrate the asset price SDE:
$$X(t + \Delta t) = X(t) + r\Delta t - \frac{1}{2}\int_t^{t+\Delta t} v(s)\,\mathrm{d}s + \int_t^{t+\Delta t} \sqrt{v(s)}\,\mathrm{d}W_S(s).$$
3. **Handle stochastic integrals**: The integral $\int_t^{t+\Delta t} \sqrt{v(s)}\,\mathrm{d}W_S(s)$ requires special treatment due to correlation. Using the orthogonal decomposition:
$$\mathrm{d}W_S(t) = \rho\,\mathrm{d}W_v(t) + \sqrt{1-\rho^2}\,\mathrm{d}W_\perp(t),$$
where $W_\perp(t)$ is independent of $W_v(t)$, the integral can be expressed as:
$$\int_t^{t+\Delta t} \sqrt{v(s)}\,\mathrm{d}W_S(s) = \rho\int_t^{t+\Delta t} \sqrt{v(s)}\,\mathrm{d}W_v(s) + \sqrt{1-\rho^2}\int_t^{t+\Delta t} \sqrt{v(s)}\,\mathrm{d}W_\perp(s).$$
4. **Approximate integrated variance**: The integrated variance $\int_t^{t+\Delta t} v(s)\,\mathrm{d}s$ and correlated integral are approximated using the trapezoidal rule or moment-matching techniques.

For computational efficiency, the algorithm uses a moment-matching approach that preserves the exact marginal distribution of the variance whilst approximating the integrated variance term.

Monte Carlo Option Pricing

Given M simulated paths of the asset price process $\{S^{(m)}(t)\}_{m=1}^M$, the price of a European call option with strike K and maturity T is:

$$C(0) = e^{-rT}\mathbb{E}\left[\max(S(T) - K, 0)\right] \approx e^{-rT}\frac{1}{M}\sum_{m=1}^{M}\max(S^{(m)}(T) - K, 0),$$

with Monte Carlo standard error:

$$\mathrm{SE} = \frac{e^{-rT}}{\sqrt{M}}\sqrt{\frac{1}{M-1}\sum_{m=1}^{M}\left(\max(S^{(m)}(T) - K, 0) - \overline{C}\right)^2},$$

where $\overline{C}$ is the sample mean of the payoffs.

For path-dependent options such as Asian or barrier options, the entire path $\{S^{(m)}(t_i)\}_{i=0}^N$ is required, making exact simulation particularly valuable for maintaining accuracy across time steps.

Calibration and Parameter Estimation

The Heston model parameters are typically calibrated to market-implied volatility surfaces by minimising the sum of squared errors between model-implied and market-observed option prices:

$$\min_{\kappa,\theta,\sigma_v,\rho,v(0)} \sum_{i,j}\left(\sigma_{\text{market}}(K_i, T_j) - \sigma_{\text{Heston}}(K_i, T_j; \kappa, \theta, \sigma_v, \rho, v(0))\right)^2,$$

where $\sigma_{\text{market}}(K_i, T_j)$ denotes the market-implied volatility for strike K_i and maturity T_j, and σ_{Heston} is computed via Fourier inversion using the characteristic function.

Typical parameter ranges observed in market calibration include:

- Mean reversion: $\kappa \in [1, 10]$
- Long-term variance: $\theta \in [0.01, 0.25]$ (corresponding to volatility of 10% to 50%)
- Volatility of volatility: $\sigma_v \in [0.1, 1.0]$
- Correlation: $\rho \in [-0.9, -0.3]$ (negative correlation capturing leverage effect)
- Initial variance: $v(0) \in [0.01, 0.25]$

Data Requirements

- **Market data**: Risk-free interest rates (e.g., LIBOR, OIS curves), spot asset prices, option market prices or implied volatility surfaces for calibration.
- **Model parameters**: Heston parameters (κ, θ, σ_v, ρ, $v(0)$) either from historical estimation or market calibration.
- **Option specifications**: Strike prices, maturities, option types (European, Asian, barrier), for pricing and sensitivity analysis.
- **Simulation parameters**: Number of Monte Carlo paths (M), number of time steps (N), time horizon (T).
- **Market assumptions**: Dividend yields, discrete dividend schedules (if applicable).

The tool includes synthetic data generation capabilities for demonstration purposes, creating realistic parameter sets and market scenarios based on empirical observations from equity and FX markets.

Code

The following code excerpts illustrate production-quality implementations using object-oriented design principles.

Python: Heston Model with Exact Simulation

```python
import numpy as np
import pandas as pd
from typing import Tuple, Optional
from scipy.stats import ncx2
from scipy.optimize import minimize
import warnings

class HestonModel:
    def __init__(self, kappa: float, theta: float, sigma_v: float,
                 rho: float, v0: float, r: float = 0.05):
        self.kappa = kappa
        self.theta = theta
        self.sigma_v = sigma_v
        self.rho = rho
        self.v0 = v0
        self.r = r

        if 2 * kappa * theta < sigma_v ** 2:
            warnings.warn("Feller condition may be violated")
```

```
20
21      def exact_variance_step(self, v_t: float, dt: float,
22                              random_vars: np.ndarray) -> np.ndarray:
23          c = self.sigma_v ** 2 * (1 - np.exp(-self.kappa * dt)) / (4 *
                self.kappa)
24          d = 4 * self.kappa * self.theta / (self.sigma_v ** 2)
25          noncentrality = 4 * self.kappa * np.exp(-self.kappa * dt) / \
26                          (self.sigma_v ** 2 * (1 - np.exp(-self.kappa *
                                dt))) * v_t
27
28          v_next = c * ncx2.rvs(d, noncentrality, size=len(random_vars))
29          return v_next
30
31      def simulate_path_exact(self, S0: float, T: float, n_steps: int,
32                              num_paths: int, seed: Optional[int] = None)
                                    -> Tuple[np.ndarray, np.ndarray]:
33          if seed is not None:
34              np.random.seed(seed)
35
36          dt = T / n_steps
37          time_grid = np.linspace(0, T, n_steps + 1)
38
39          S = np.zeros((num_paths, n_steps + 1))
40          v = np.zeros((num_paths, n_steps + 1))
41
42          S[:, 0] = S0
43          v[:, 0] = self.v0
44
45          for i in range(n_steps):
46              Z_v = np.random.normal(0, 1, num_paths)
47              Z_S = np.random.normal(0, 1, num_paths)
48              Z_corr = self.rho * Z_v + np.sqrt(1 - self.rho ** 2) * Z_S
49
50              v_t = v[:, i]
51              v_next = self.exact_variance_step(v_t, dt, Z_v)
52
53              v_bar = (v_t + v_next) / 2
54              sqrt_v_bar = np.sqrt(np.maximum(v_bar, 0))
55
56              K0 = -self.rho * self.kappa * self.theta * dt / self.
                    sigma_v
57              K1 = (self.rho * self.kappa / self.sigma_v - 0.5) * dt -
                    self.rho / self.sigma_v
58              K2 = self.rho / self.sigma_v
59              K3 = (1 - self.rho ** 2) * dt
60
61              log_S_next = (np.log(S[:, i]) + self.r * dt +
62                            K0 + K1 * v_t + K2 * v_next +
63                            sqrt_v_bar * np.sqrt(K3) * Z_S)
64
65              S[:, i + 1] = np.exp(log_S_next)
66              v[:, i + 1] = v_next
67
68          return S, v
69
70      def characteristic_function(self, u: float, T: float, S0: float) ->
            complex:
71          i = 1j
```

```
        kappa = self.kappa
        theta = self.theta
        sigma_v = self.sigma_v
        rho = self.rho
        v0 = self.v0

        d = np.sqrt((rho * sigma_v * i * u - kappa) ** 2 + sigma_v ** 2
            * (i * u + u ** 2))
        g = (kappa - rho * sigma_v * i * u - d) / (kappa - rho *
           sigma_v * i * u + d)

        C = self.r * i * u * T + kappa * theta / (sigma_v ** 2) * \
            ((kappa - rho * sigma_v * i * u - d) * T -
             2 * np.log((1 - g * np.exp(-d * T)) / (1 - g)))

        D = (kappa - rho * sigma_v * i * u - d) / (sigma_v ** 2) * \
            (1 - np.exp(-d * T)) / (1 - g * np.exp(-d * T))

        return np.exp(C + D * v0 + i * u * np.log(S0))

    def price_european_call_fft(self, S0: float, K: float, T: float,
                                N: int = 4096, alpha: float = 1.5) ->
                                   float:
        i = 1j
        eta = 0.25
        b = N * eta / 2
        u = np.arange(1, N + 1) * eta

        kappa = np.log(K)
        v = u - (alpha + 1) * i
        phi = self.characteristic_function(v, T, S0)
        psi = np.exp(-r * T) * phi / ((alpha + i * u) * (alpha + 1 + i
           * u))

        integral = np.exp(-alpha * kappa) / np.pi * np.sum(
            np.real(psi * np.exp(-i * u * kappa)) * eta
        )

        return S0 - K * np.exp(-self.r * T) * integral

    def price_european_call_mc(self, S0: float, K: float, T: float,
                               num_paths: int = 10000, n_steps: int =
                                  252) -> Tuple[float, float]:
        S, _ = self.simulate_path_exact(S0, T, n_steps, num_paths)

        payoffs = np.maximum(S[:, -1] - K, 0)
        price = np.exp(-self.r * T) * np.mean(payoffs)
        std_error = np.exp(-self.r * T) * np.std(payoffs) / np.sqrt(
           num_paths)

        return price, std_error

class HestonOptionPricer:
    def __init__(self, model: HestonModel):
        self.model = model

    def price_asian_call(self, S0: float, K: float, T: float,
```

```
                        num_paths: int = 10000, n_steps: int = 252) ->
                            Tuple[float, float]:
        S, _ = self.model.simulate_path_exact(S0, T, n_steps, num_paths
            )

        asian_price = np.mean(S, axis=1)
        payoffs = np.maximum(asian_price - K, 0)
        price = np.exp(-self.model.r * T) * np.mean(payoffs)
        std_error = np.exp(-self.model.r * T) * np.std(payoffs) / np.
            sqrt(num_paths)

        return price, std_error

    def price_barrier_up_and_out_call(self, S0: float, K: float, B:
        float, T: float,
                                      num_paths: int = 10000, n_steps:
                                          int = 252) -> Tuple[float,
                                          float]:
        S, _ = self.model.simulate_path_exact(S0, T, n_steps, num_paths
            )

        max_price = np.max(S, axis=1)
        knock_out = max_price >= B
        payoffs = np.maximum(S[:, -1] - K, 0)
        payoffs[knock_out] = 0

        price = np.exp(-self.model.r * T) * np.mean(payoffs)
        std_error = np.exp(-self.model.r * T) * np.std(payoffs) / np.
            sqrt(num_paths)

        return price, std_error
```

This implementation ensures numerical stability through exact variance sampling and provides a foundation for pricing various exotic options.

Python: Streamlit Interactive Dashboard

```
import streamlit as st
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from heston_model import HestonModel, HestonOptionPricer

st.set_page_config(
    page_title="Heston Stochastic Volatility Dashboard",
    page_icon="      ",
    layout="wide"
)

st.title("Heston Stochastic Volatility Model with Exact Simulation")

with st.sidebar:
    st.header("Model Parameters")
    kappa = st.slider("Mean Reversion (  )", 0.5, 10.0, 2.0, 0.1)
    theta = st.slider("Long-term Variance (  )", 0.01, 0.25, 0.04,
        0.01)
```

```
    sigma_v = st.slider("Vol of Vol (     )", 0.1, 1.0, 0.3, 0.05)
    rho = st.slider("Correlation (  )", -0.9, 0.9, -0.7, 0.1)
    v0 = st.slider("Initial Variance ( v  )", 0.01, 0.25, 0.04, 0.01)

    st.header("Market Parameters")
    S0 = st.number_input("Initial Stock Price", 50, 200, 100)
    r = st.slider("Risk-Free Rate (%)", 0.0, 10.0, 5.0) / 100

    st.header("Simulation Settings")
    num_paths = st.slider("Monte Carlo Paths", 1000, 50000, 10000)
    n_steps = st.slider("Time Steps", 50, 500, 252)
    T = st.slider("Time Horizon (years)", 0.1, 2.0, 1.0, 0.1)

    st.header("Option Parameters")
    K = st.number_input("Strike Price", 50, 200, 100)
    option_type = st.selectbox("Option Type",
                               ["European Call", "Asian Call", "Barrier
                                   Up-and-Out Call"])
    if option_type == "Barrier Up-and-Out Call":
        B = st.number_input("Barrier Level", S0, S0 * 2, int(S0 * 1.5))

model = HestonModel(kappa, theta, sigma_v, rho, v0, r)
pricer = HestonOptionPricer(model)

col1, col2, col3, col4 = st.columns(4)
with col1:
    st.metric("Mean Reversion Speed", f"{kappa:.2f}")
with col2:
    st.metric("Long-term Volatility", f"{np.sqrt(theta)*100:.2f}%")
with col3:
    st.metric("Vol of Vol", f"{sigma_v:.2f}")
with col4:
    st.metric("Correlation", f"{rho:.2f}")

if st.button("Run Simulation", type="primary"):
    with st.spinner("Running Monte Carlo simulation..."):
        S, v = model.simulate_path_exact(S0, T, n_steps, num_paths)

        if option_type == "European Call":
            price, std_err = pricer.model.price_european_call_mc(S0, K,
                T, num_paths, n_steps)
        elif option_type == "Asian Call":
            price, std_err = pricer.price_asian_call(S0, K, T,
               num_paths, n_steps)
        else:
            price, std_err = pricer.price_barrier_up_and_out_call(S0, K
               , B, T, num_paths, n_steps)

        st.subheader("Pricing Results")
        result_col1, result_col2, result_col3 = st.columns(3)
        with result_col1:
            st.metric("Option Price", f"${price:.4f}")
        with result_col2:
            st.metric("Standard Error", f"${std_err:.4f}")
        with result_col3:
            st.metric("95% Confidence Interval",
                      f"[${price-1.96*std_err:.4f}, ${price+1.96*std_err
                          :.4f}]")
```

```
        time_grid = np.linspace(0, T, n_steps + 1)

        st.subheader("Simulated Price Paths")
        fig_paths = go.Figure()

        sample_paths = min(100, num_paths)
        for i in range(sample_paths):
            fig_paths.add_trace(go.Scatter(
                x=time_grid,
                y=S[i, :],
                mode='lines',
                line=dict(width=0.5, color='rgba(104, 101, 242, 0.1)'),
                showlegend=False
            ))

        mean_path = np.mean(S, axis=0)
        std_path = np.std(S, axis=0)

        fig_paths.add_trace(go.Scatter(
            x=time_grid,
            y=mean_path,
            mode='lines',
            name='Mean Path',
            line=dict(width=2, color='#6865F2')
        ))

        fig_paths.add_trace(go.Scatter(
            x=time_grid,
            y=mean_path + 1.96 * std_path,
            mode='lines',
            name='95% Upper Bound',
            line=dict(width=1, color='#5DFFBC', dash='dash'),
            showlegend=True
        ))

        fig_paths.add_trace(go.Scatter(
            x=time_grid,
            y=mean_path - 1.96 * std_path,
            mode='lines',
            name='95% Lower Bound',
            line=dict(width=1, color='#5DFFBC', dash='dash'),
            fill='tonexty',
            fillcolor='rgba(93, 255, 188, 0.1)'
        ))

        fig_paths.update_layout(
            title="Simulated Asset Price Paths",
            xaxis_title="Time (years)",
            yaxis_title="Asset Price",
            hovermode='x unified',
            height=400
        )
        st.plotly_chart(fig_paths, use_container_width=True)

        st.subheader("Simulated Variance Paths")
        fig_var = go.Figure()
```

```
        for i in range(sample_paths):
            fig_var.add_trace(go.Scatter(
                x=time_grid,
                y=v[i, :],
                mode='lines',
                line=dict(width=0.5, color='rgba(255, 107, 107, 0.1)'),
                showlegend=False
            ))

        mean_var = np.mean(v, axis=0)

        fig_var.add_trace(go.Scatter(
            x=time_grid,
            y=mean_var,
            mode='lines',
            name='Mean Variance',
            line=dict(width=2, color='#FF6B6B')
        ))

        fig_var.add_hline(y=theta, line_dash="dash", line_color="gray",
                          annotation_text=f"Long-term Variance (  ={
                              theta:.4f})")

        fig_var.update_layout(
            title="Simulated Variance Paths",
            xaxis_title="Time (years)",
            yaxis_title="Variance",
            hovermode='x unified',
            height=400
        )
        st.plotly_chart(fig_var, use_container_width=True)

        st.subheader("Distribution Analysis")
        dist_col1, dist_col2 = st.columns(2)

        with dist_col1:
            fig_hist_price = go.Figure()
            fig_hist_price.add_trace(go.Histogram(
                x=S[:, -1],
                nbinsx=50,
                marker_color='#6865F2',
                opacity=0.7,
                name='Final Prices'
            ))
            fig_hist_price.add_vline(x=K, line_dash="dash", line_color=
                "red",
                                     annotation_text=f"Strike (K={K})")
            fig_hist_price.update_layout(
                title="Distribution of Final Asset Prices",
                xaxis_title="Asset Price",
                yaxis_title="Frequency",
                height=300
            )
            st.plotly_chart(fig_hist_price, use_container_width=True)

        with dist_col2:
            fig_hist_var = go.Figure()
            fig_hist_var.add_trace(go.Histogram(
```

```
                x=v[:, -1],
                nbinsx=50,
                marker_color='#FF6B6B',
                opacity=0.7,
                name='Final Variances'
            ))
            fig_hist_var.add_vline(x=theta, line_dash="dash",
               line_color="gray",
                                   annotation_text=f"Long-term (  ={
                                      theta:.4f})")
            fig_hist_var.update_layout(
                title="Distribution of Final Variances",
                xaxis_title="Variance",
                yaxis_title="Frequency",
                height=300
            )
            st.plotly_chart(fig_hist_var, use_container_width=True)

        st.subheader("Convergence Analysis")
        convergence_paths = [1000, 2500, 5000, 7500, 10000, 15000,
           20000, 30000, 50000]
        convergence_paths = [p for p in convergence_paths if p <=
           num_paths]

        prices_conv = []
        std_errs_conv = []

        for n_paths in convergence_paths:
            if option_type == "European Call":
                p, se = pricer.model.price_european_call_mc(S0, K, T,
                   n_paths, n_steps)
            elif option_type == "Asian Call":
                p, se = pricer.price_asian_call(S0, K, T, n_paths,
                   n_steps)
            else:
                p, se = pricer.price_barrier_up_and_out_call(S0, K, B,
                   T, n_paths, n_steps)
            prices_conv.append(p)
            std_errs_conv.append(se)

        fig_conv = make_subplots(
            rows=2, cols=1,
            subplot_titles=("Price Convergence", "Standard Error
               Convergence"),
            vertical_spacing=0.1
        )

        fig_conv.add_trace(
            go.Scatter(x=convergence_paths, y=prices_conv, mode='lines+
               markers',
                       name='Option Price', line=dict(color='#6865F2',
                          width=2)),
            row=1, col=1
        )
        fig_conv.add_hline(y=price, line_dash="dash", line_color="gray"
           ,
                           annotation_text="Final Price", row=1, col=1)

```

```
        fig_conv.add_trace(
            go.Scatter(x=convergence_paths, y=std_errs_conv, mode='
                lines+markers',
                       name='Standard Error', line=dict(color='#FF6B6B',
                           width=2)),
            row=2, col=1
        )

        fig_conv.update_xaxes(title_text="Number of Paths", row=2, col
            =1)
        fig_conv.update_yaxes(title_text="Price", row=1, col=1)
        fig_conv.update_yaxes(title_text="Standard Error", row=2, col
            =1)
        fig_conv.update_layout(height=600, showlegend=False)

        st.plotly_chart(fig_conv, use_container_width=True)
```

This dashboard provides comprehensive visualisation of volatility dynamics, option pricing, and Monte Carlo convergence behaviour.

Expected Outputs

- Accurate Monte Carlo option prices for European, Asian, and barrier options using exact simulation methods.
- Interactive visualisations of simulated asset price and variance paths demonstrating mean reversion and volatility clustering.
- Distributional analysis of final asset prices and variances, showing the impact of stochastic volatility on option payoffs.
- Convergence analysis demonstrating how Monte Carlo estimates stabilise as the number of paths increases.
- Real-time sensitivity analysis showing how option prices respond to changes in model parameters (mean reversion, vol-of-vol, correlation).
- Comparison between exact simulation and Euler-Maruyama schemes, highlighting the importance of numerical stability.
- Professional dashboard interface suitable for quantitative research, risk management, and educational demonstrations.

Interview Questions

1. Explain why exact simulation is preferred over Euler-Maruyama discretisation for the Heston model. What issues arise when the Feller condition is violated, and how does exact simulation handle this?
2. How does the correlation parameter ρ affect option prices in the Heston model? Discuss the leverage effect and its empirical justification.
3. Describe the characteristic function approach to option pricing. How would you extend this framework to price path-dependent options such as Asian or lookback options?
4. What are the computational trade-offs between Monte Carlo simulation and Fourier inversion methods for pricing European options? When would you choose one over the other?

5. How would you calibrate the Heston model to market data? Discuss the challenges of parameter estimation and potential model misspecification.

6. Explain the relationship between the mean reversion parameter κ and volatility clustering. How does this parameter affect the distribution of realised volatility over different time horizons?

Resume Bullets

- Implemented exact simulation scheme for the Heston stochastic volatility model using non-central chi-squared sampling, eliminating discretisation bias and ensuring numerical stability in Monte Carlo option pricing.
- Developed interactive Streamlit dashboard with real-time visualisation of volatility dynamics, option pricing sensitivities, and Monte Carlo convergence analysis for quantitative research applications.
- Designed object-oriented Python framework for pricing European and exotic options (Asian, barrier) under stochastic volatility, incorporating characteristic function methods for validation and calibration.
- Created comprehensive Monte Carlo pricing engine with exact variance simulation, enabling accurate pricing of path-dependent derivatives whilst maintaining computational efficiency through vectorised NumPy operations.